

Optimization using Gradient Descent Algorithms

The Path to the Valley Floor: A Story-Driven Guide to Optimization

1. The Quest for the Global Minimum

Imagine you are standing on a mountain peak shrouded in a thick, blinding fog. Your objective is simple but high-stakes: you must find your way to the absolute bottom of the valley. Because of the fog, you cannot see the landscape; you can only feel the tilt of the ground beneath your boots. In machine learning, this journey is known as **Optimization**, and the valley floor you are searching for is the **Global Minimum**.

The landscape you are navigating is the "Error Bowl." In a perfect scenario, this bowl would be as smooth as a polished ceramic cereal bowl, leading you directly to the center. However, real-world data creates an Error Bowl filled with "divots and dents." These tiny dips are **Local Minima**—traps that look like the bottom but are actually just shallow pits on the side of the mountain.

Global Minimum vs. Local Minima

- **Global Minimum:** The absolute lowest point of the entire error landscape; the state where the model's error is at its absolute theoretical lowest.
- **Local Minima:** Shallow "dents" or "divots" in the landscape that can trick a model into thinking it has reached the bottom when it is actually stuck in a sub-optimal spot.

Optimization is the "steering" mechanism of artificial intelligence. By calculating which way is "downhill" and taking a step, we reduce the model's error. Reaching the bottom means our model has learned the patterns of the data with the highest possible accuracy. To get there, however, we need to understand the mathematical engine that powers our descent.

2. The Math Engine: Why the "Square" Disappears

To move down the mountain, we perform "Math Surgery" on our error. If we are trying to minimize **Mean Squared Error (MSE)**, you might notice that the "square" seems to vanish when we actually update our weights. This isn't magic; it is the result of fundamental calculus.

When we calculate the **Gradient** (the slope), we apply the **Power Rule** and the **Chain Rule** to our Loss function. The square in $(wx - y)^2$ is "demoted" to a power of 1 as it moves to the front of the equation as a multiplier.

Feature	The Loss Function (The Height)	The Gradient (The Slope)
Purpose	Tells us how "bad" the model is currently performing.	Tells us which direction and how far to move the "knob."
Formula	$\text{Loss} = (wx - y)^2$	$\text{Gradient} = 2 \cdot (wx - y) \cdot x$
The "2"	The exponent representing the "squared" error.	Moved to the front via the Power Rule .
The "Square"	Present; it punishes large errors heavily.	Disappears; demoted to a power of 1.

Architect's Perspective: We do not use the squared error to update our weights because squaring a negative error makes it positive. If we used the Loss instead of the Gradient, we would lose our "compass"—we wouldn't know if our guess was too high or too low! The Gradient preserves the direction (the sign) of the error, ensuring we always step toward the floor.

3. The Three Hikers: Batch, Stochastic, and Mini-Batch

There are three different types of "hikers" we can send down the mountain. Each uses the math engine with a different philosophy.

- **Batch Gradient Descent (BGD): The Slow & Steady Heavy Marble.** BGD is like a group of hikers who refuse to move until every single person has agreed on the direction. It calculates the average gradient for the entire dataset before taking one very calculated step.
- **Stochastic Gradient Descent (SGD): The Caffeinated Ping-Pong Ball.** Also known as the "Drunk Walker," SGD doesn't wait for a consensus. It picks one random data point, treats it as the only point in the world, and bolts in that direction immediately.
- **Mini-Batch SGD: The "Goldilocks" Middle Ground.** This is the industry standard. It looks at a small group (a "batch") of points—typically 32 to 128—to find a stable consensus before moving.

The Superpower of Noise

While SGD's path looks chaotic, its jittery movement is actually a superpower. Because it bounces around, it has enough energy to "pop" out of the shallow local minima that would trap a heavy, polite Batch marble. This "noise" allows the model to ignore minor quirks in the data and find a more robust solution—a concept known as **Generalization**.

Feature	Batch (BGD)	Stochastic (SGD)	Mini-Batch
Data per Step	Entire dataset	1 random sample	A small group (e.g., 32)
Path to Minimum	Smooth and direct	Noisy and "drunk-looking"	Stable but fast
Memory Usage	High (Needs all data)	Very Low (Needs 1 point)	Adjustable

4. The Lab Experiment: A Single Epoch Showdown

Let's test these hikers on a "*Line Fitting*" experiment.

We want to find the weight (w) for the model $\hat{y} = w \cdot x$.

- **Dataset:** (1,1), (2,2), (3,3)
(*Optimal* w is 1.0 because the ideal equation for these points will be $y = x$)
- **Starting Point/Initial weight:** $w = 0.5$;
- **Learning Rate (α):** 0.1

Comparative Analysis: Consensus vs. Impulse

Path A: The Batch "Consensus"

Batch GD looks at all $n = 3$ points and averages them.

Epoch-1: $w = 0.5$

Point	Error ($w * x - y$)
(1, 1)	$(0.5) * (1) - (1) = -0.5$
(2, 2)	$(0.5) * (2) - (2) = -1.0$
(3, 3)	$(0.5) * (3) - (3) = -1.5$

$$\text{Averaged Gradient} = \frac{2}{3} [(-0.5 \cdot 1) + (-1.0 \cdot 2) + (-1.5 \cdot 3)] = \frac{2}{3} [-7.0] = -4.66667$$

$$\text{Update: } w = 0.5 - (0.1 \cdot -4.66667) = 0.96667$$

Similarly repeat for next iteration using $w = 0.96667$

Path B: The Stochastic "Impulse"

SGD updates after *every* point ($n=1$), meaning we divide by 1 (or simply multiply by 2).

Step	Point Used	Calculation ($2 \cdot x \cdot (wx - y)$)	New Weight (w)
1	(1,1)	$2 \cdot 1 \cdot (0.5 \cdot 1 - 1) = -1.0$	$0.5 - (0.1 \cdot -1.0) = 0.6$
2	(2,2)	$2 \cdot 2 \cdot (0.6 \cdot 2 - 2) = -3.2$	$0.6 - (0.1 \cdot -3.2) = 0.92$
3	(3,3)	$2 \cdot 3 \cdot (0.92 \cdot 3 - 3) = -1.44$	$0.92 - (0.1 \cdot -1.44) = 1.064$

Note on Exploding Gradients: Notice we divide by the number of points (n). In Batch GD, if we had 1 million points and *didn't* divide by $n=1,000,000$, the gradient would be so massive that the weight update would "explode," sending w to *infinity* and crashing the model.

5. The Marathon: Convergence and Multiple Epochs

One lap (*epoch*) is just the beginning. We stop training only when we reach **Convergence** — *the point where the weights stop changing meaningfully*. We use three signals:

1. **The Tolerance Threshold (ϵ):** The change in w is tiny (e.g., < 0.0001).
2. **The Loss Plateau:** The error bowl height stops decreasing.
3. **Maximum Epochs:** A safety net to prevent infinite loops.

Batch Gradient Descent Iteration Table

Epoch	w_{start}	Avg Grad (G)	w_{new}	Change (Δw)
1	0.50000	-4.66667	0.96667	0.46667
2	0.96667	-0.31111	0.99778	0.03111
3	0.99778	-0.02074	0.99985	0.00207
4	0.99985	-0.00138	0.99999	0.00014
5	0.99999	-0.00009	1.00000	0.00001

Mini-Batch SGD Iteration Table (Batch Size = 2)

In Mini-Batch, we see the weight update twice per epoch: once after the first "consensus" of 2 points, and once after the remaining point.

Epoch	w_{start}	After Batch 1 (P_1, P_2)	After Batch 2 (P_3)	Final w (Epoch)
1	0.50000	0.75000	1.20000	1.20000
2	1.20000	1.10000	0.92000	0.92000
3	0.92000	0.96000	1.03200	1.03200
4	1.03200	1.01600	0.98720	0.98720
5	0.98720	0.99360	1.00512	1.00512

6. The Real-World Verdict: Choosing Your Driver

In our small lab, Batch GD hit the target of 1.0 most gracefully. But in production, Batch GD hits a **"Memory Wall."** Imagine trying to drink an entire ocean in one gulp; that is Batch GD trying to load 100GB of data into 16GB of RAM. The system simply crashes.

Mini-Batch SGD is the industry standard for models like ChatGPT because it balances accuracy with the ability to use hardware (GPUs) to process data in parallel chunks.

The Decision Matrix

If your dataset is...	And your memory is...	Use this Method	The Path
Small (Hundreds of rows)	Plentiful	Batch GD	Smooth and direct
Massive (Millions of rows)	Limited	SGD	Noisy "Drunk Walk"
Standard / Large	Modern GPU	Mini-Batch SGD	Stabilized zig-zag

Final Summary: **The Learning Rate Schedule.**

As you can see in the tables for SGD and Mini-Batch, the weights "dance" around 1.0 (e.g., 1.00512). To stop the Drunk Walker from overshooting the goal forever, we use a **Learning Rate Schedule**.

- We start fast to cover ground, then "throttle down" the speed (α) as we get closer to the bottom.
- This allows the model to settle precisely at the Global Minimum, turning a chaotic descent into a perfect landing.

Practice Question #1: (Optimizing Multiple parameters)

Optimize the weights and bias for the linear regression model defined by $y = w \cdot x + b$ over the set of points (x, y) : $[(1, 1), (2, 3), (3, 4)]$. Consider the Loss function defined on MSE (Mean Squared Error). Assume learning rate $\alpha = 0.1$ and initial guess $w = 0.5$ and $b = 0.5$

Solve using:

- Batch Gradient Descent
- Stochastic Gradient Descent
- Mini-Batch Gradient Descent

Also, compare the outcomes of these 3 algorithms after 1 epoch.

Practice Question #2:

Optimize the weights and bias for the linear regression model defined by $y = w \cdot x$ over the set of points (x, y) : $[(1, 1), (2, 2), (3, 3)]$. Consider the Loss function L defined on **MAE (Mean Absolute Error)**.

The loss function for a single point is:

$$L = |wx - y|$$

Assume learning rate $\alpha = 0.1$ and initial guess $w = 0.5$

Solve using:

- Batch Gradient Descent
- Stochastic Gradient Descent
- Mini-Batch Gradient Descent

Hint:

To find the gradient, we use the derivative of an absolute value.

The derivative of $|u|$ is the **sign function**:

- Derivative is $+1$ if u is positive, and
- Derivative is -1 if u is negative.

Using the Chain Rule:

$$\frac{\partial L}{\partial w} = \text{sign}(wx - y) \cdot x$$
